Parcelo: Architecture and Design Blueprint

The build plan for Parcelo, a logistics platform for booking, orders, shipments, dispatch, last-mile delivery, fleet and inventory

Version 2.0 | 10 November 2026

Parcelo is three small React apps (customer, ops dashboard and driver) that share one Firebase backend, built in three phases: **Phase 1** lets customers book and track deliveries, **Phase 2** adds dispatch and drivers, and **Phase 3** adds inventory, messages and reports. This document explains how it is put together, what every screen does, and exactly what to build in each phase.

1. The Big Picture

1.1 What Parcelo does

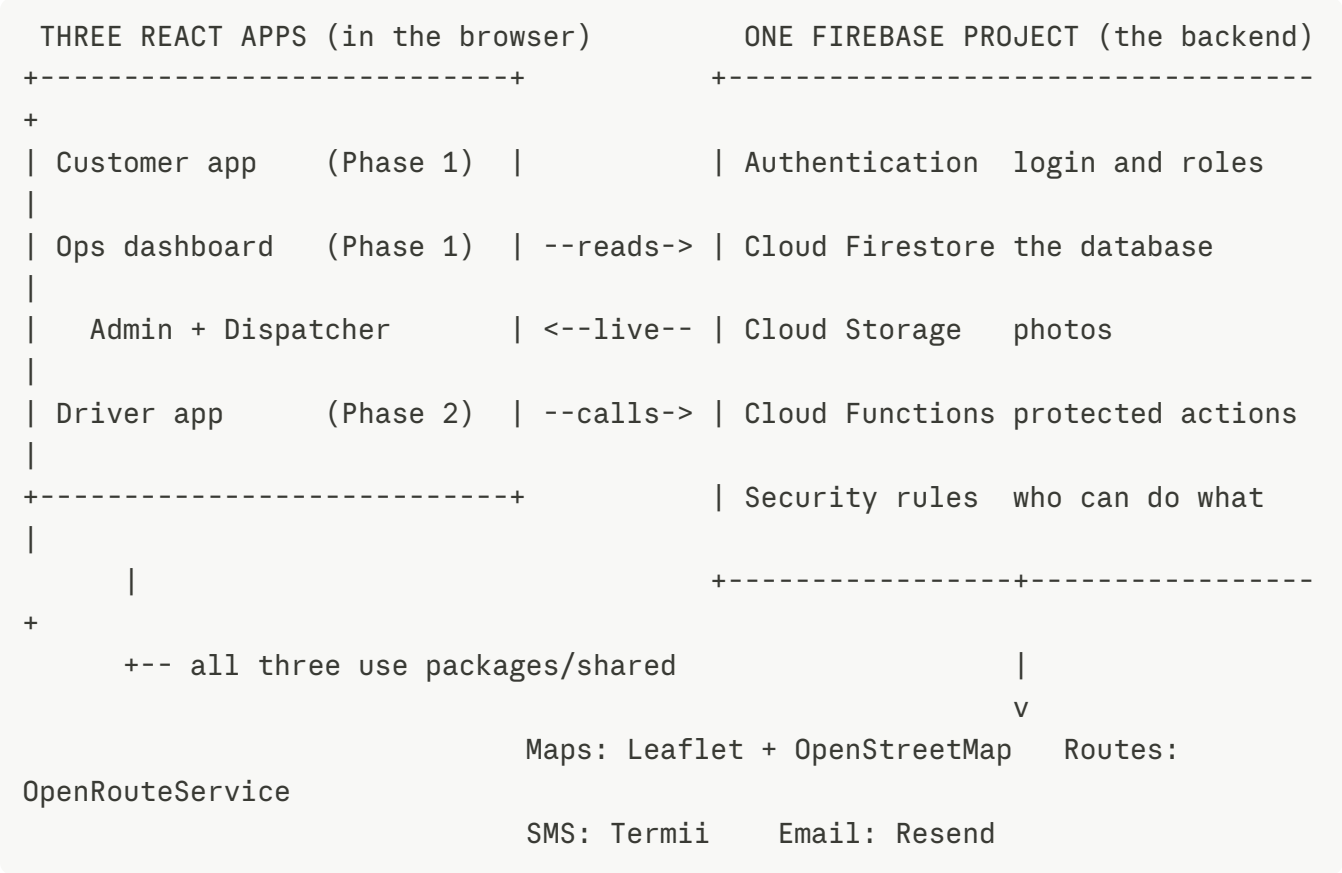
A customer books a delivery and sees the price. The business assigns a driver and vehicle. The driver delivers and uploads proof. Everyone can follow the package with a tracking number.

1.2 Who uses it, and which app they use

Person	What they do	Their app	Starts in
Customer	Books deliveries, tracks packages	Customer app	Phase 1
Anyone (no login)	Looks up a tracking number	Customer app (public tracking page)	Phase 1
Admin	Runs the business, updates shipments, manages staff and stock	Ops dashboard (all menus)	Phase 1
Dispatcher	Assigns jobs to drivers and vehicles	Ops dashboard (dispatch menus only)	Phase 2
Driver	Delivers packages, updates status	Driver app (on a phone)	Phase 2

The three apps are separate React websites. Each has its own folder and its own web address, but they share one Firebase backend and one folder of shared code.

1.3 How the pieces fit together



Reading it: the apps read data straight from the Firestore database, and the data updates live on screen. Anything that must be protected, such as booking, assigning a driver, changing a status or reducing stock, goes through a Cloud Function that checks who you are and applies the rules. Security rules are the last line of defence on every read and write.

1.4 The journey of one package

1. The customer fills in pickup, drop-off and package details and sees a price. *(Phase 1)*
2. The customer confirms. The system creates the **order** and a **shipment** with a tracking number and the status *Pending*. *(Phase 1)*
3. The dispatcher assigns a driver and vehicle: *Assigned*. *(Phase 2)*
4. The driver picks up the package: *In Transit*, then *Out for Delivery*. *(Phase 2)*
5. The driver delivers and uploads a photo: *Delivered*. *(Phase 1 by admin, Phase 2 by driver)*
6. The customer follows every step on the tracking page. *(Phase 1)*
7. Stock goes down when product orders are placed, and messages are sent at each step. *(Phase 3)*

2. Foundations (Shared by All Phases)

2.1 Tech stack

Part	Choice	Why
The three apps	React with Vite	Fast to build, made of reusable components
Page switching	React Router	One address per screen in each app
Shared state	React Context	Holds who is logged in and their role
Styling	Tailwind CSS with the Parcelo colour variables	Quick, consistent look
Login	Firebase Authentication (email and password)	Ready-made, safe password handling
Roles	Firebase custom claims (customer, admin, dispatcher, driver)	Role travels with the login
Database	**Cloud Firestore**	Stores all shared data; updates appear live
Photos	Firebase Storage	Proof-of-delivery images
Protected actions	Cloud Functions (Node.js)	Booking, assigning, status changes, stock, messages
Access rules	Firestore and Storage security rules	Decide who can read and write what
Maps	Leaflet with OpenStreetMap	Free, no key
Charts	Recharts (Ops dashboard, Phase 3)	Simple dashboard charts
Icons	lucide-react	One consistent icon set
Messages	Termii (SMS), Resend (email), called from Cloud Functions	Phase 3
Hosting	Firebase Hosting, one site per app	Three addresses, one project

The database is Cloud Firestore. It is a NoSQL document database: data is kept as *documents* (like a record with named fields) inside *collections* (like a folder of records). It has no joins, so small details such as the customer's name are copied onto an order when

it is created.

What "the server" means in this document: there is no server you run. Wherever the plan says "the server", it means the Cloud Functions and security rules in the Firebase project.

Cost note: Cloud Functions need Firebase's pay-as-you-go (Blaze) plan. It has a free monthly allowance that a project this size normally stays within, but a payment card is needed to switch it on.

2.2 Project structure: three apps, one shared folder, one backend

Everything lives in one repository (a *monorepo*) with three kinds of folder: the three apps, the shared code, and the Firebase backend.

```
parcelo/
  apps/
    customer/      Customer app and public tracking page        (Phase 1)
    ops/           Ops dashboard for Admin and Dispatcher       (Phase 1,
grows in 2 and 3)
    driver/        Driver app, installable on a phone           (Phase 2)
  packages/
    shared/        code used by all three apps                  (Phase 1)
  firebase/        the backend: Cloud Functions and rules       (Phase 1)
  firebase.json    says which folder is deployed to which website
  .firebaserc      names the Firebase project and its three hosting sites
  package.json     lets the apps import the shared folder
  README.md
```

Every app has the same skeleton, so once one is built the others are quick:

```
apps/<app-name>/
  index.html          the single page the app loads into
  vite.config.js      build settings
  .env                Firebase keys and the web addresses of the other apps
  public/             favicon and images
  src/
    main.jsx          starts the app
    App.jsx           lists every route (address) of this app
    layouts/          the frame around pages (top bar, sidebar)
    pages/            one file per screen
    components/       pieces used only by this app
    hooks/            small reusable logic for this app
    api/              functions that read data or call Cloud Functions for
this app
```

Customer app (`apps/customer`)

```
apps/customer/src/
  layouts/
    PublicLayout.jsx            top bar for visitors
    CustomerLayout.jsx          top bar for logged-in customers
  pages/
    HomePage.jsx                tracking box and Book button
(P1)
    TrackingPage.jsx            /track/:trackingNumber
(P1)
    LoginPage.jsx, RegisterPage.jsx
(P1)
    BookingPage.jsx             holds the three booking steps
(P1)
    MyOrdersPage.jsx            list of the customer's orders
(P1)
    OrderDetailsPage.jsx        price, timeline, rider; live map in P2
(P1, P2)
    NotificationSettingsPage.jsx   SMS or email choice
(P3)
  components/
    booking/   PickupDropoffStep.jsx, PackageStep.jsx, QuoteConfirmStep.jsx
    orders/    OrderCard.jsx, OrderTimeline.jsx
    tracking/  TrackingSearch.jsx, TrackingTimeline.jsx
    driver/    DriverCard.jsx, LiveMap.jsx
(P2)
  hooks/       useQuote.js, useMyOrders.js, useTracking.js, useLiveDriver.js
(P2)
  api/         quotes.js (calls getQuote), orders.js (calls bookDelivery,
reads own orders),
               tracking.js (reads publicTracking)
```

Ops dashboard (`apps/ops`): one app for Admin and Dispatcher

Admins and dispatchers use the same app. The sidebar is built from the person's role: an **Admin** sees every link, a **Dispatcher** sees only Dispatch board, Shipments, Drivers and Fleet. After login an admin lands on `/` and a dispatcher lands on `/dispatch`. Hiding a link is only a convenience; the security rules still decide what each role can read and write.

In Phase 1 only the Admin pages exist, and the admin does the dispatcher's job by hand. The dispatch pages arrive in Phase 2. They live in their own folders, so they can be split into a separate app later if the team grows.

```
apps/ops/src/
  layouts/
    OpsLayout.jsx               dark sidebar and top bar; the links depend on
the role
  pages/
    DashboardPage.jsx           counts by status
(P1, Admin)
    OrdersPage.jsx              all orders, filter and search
(P1, Admin)
    ShipmentPage.jsx            status button, rider, photo, payment
(P1, Admin)
    DispatchBoardPage.jsx       waiting jobs, free drivers, map
(P2, Dispatcher and Admin)
    ShipmentsPage.jsx           all shipments with filters
(P2)
    DriversPage.jsx             availability and last position; Admin can add
and edit   (P2)
    FleetPage.jsx               vehicle status; Admin can add and edit
(P2)
    UsersPage.jsx               create dispatcher and driver accounts
(P2, Admin)
    InventoryPage.jsx, ProductPage.jsx
(P3, Admin)
    ReportsPage.jsx             cards and charts
(P3, Admin)
    NotificationLogPage.jsx     messages sent
(P3, Admin)
    SettingsPage.jsx            prices and reorder defaults
(P3, Admin)
  components/
    orders/     OrderTable.jsx, PaymentStatusSelect.jsx
(P1)
    shipments/  StatusButtons.jsx, ProofUpload.jsx, RiderForm.jsx
(P1)
    dispatch/   UnassignedList.jsx, DriverPicker.jsx, VehiclePicker.jsx,
                AssignModal.jsx, ReassignModal.jsx
(P2)
    map/        LiveMapPanel.jsx, RoutePreview.jsx
(P2)
    drivers/    DriverTable.jsx, DriverForm.jsx
(P2)
    fleet/      VehicleTable.jsx, VehicleForm.jsx, VehicleStatusSelect.jsx
(P2)
    inventory/  ProductTable.jsx, ProductForm.jsx, StockAdjustModal.jsx,
                LowStockBanner.jsx
(P3)
```

```
    reports/    SummaryCard.jsx, OrdersPerWeekChart.jsx
(P3)
  hooks/        useAllOrders.js, useShipment.js
(P1)
                useUnassigned.js, useAvailableDrivers.js,
                useAvailableVehicles.js
(P2)
                useProducts.js, useReport.js
(P3)
  api/          orders.js, shipments.js (calls updateShipmentStatus,
saveProof)   (P1)
                dispatch.js (calls assignShipment, reassignShipment),
drivers.js,
                vehicles.js, staff.js (calls createStaffAccount)
(P2)
                products.js, reports.js
(P3)
```

Driver app (`apps/driver`)

```
apps/driver/
  public/
    manifest.webmanifest        name, icon and colours so the app can be
installed
    icons/                      home-screen icons
  src/
    service-worker.js           keeps the app opening when the signal is weak
    layouts/
      DriverLayout.jsx          top bar and a bottom bar with three buttons
    pages/
      LoginPage.jsx
      MyJobsPage.jsx            today's jobs
      JobDetailsPage.jsx        map, big status buttons, photo
      ProfilePage.jsx           available or off duty
    components/
      JobCard.jsx, JobStatusButtons.jsx (large), ProofCamera.jsx,
      LocationSharingToggle.jsx, OfflineBanner.jsx, InstallPrompt.jsx
    hooks/     useMyJobs.js, useShareLocation.js, useOfflineQueue.js
    api/       jobs.js (reads jobs, calls updateShipmentStatus, saveProof),
               location.js (writes the driver's own position)
```

All driver files belong to Phase 2.

Shared folder (`packages/shared`): the files that connect the apps

```
packages/shared/
  package.json
  src/
    firebase/
      firebaseApp.js       starts Firebase once, from the keys in .env
      auth.js              register, login, logout, read the user's role
      firestore.js         the database handle and small helpers
      functions.js         one helper for calling any Cloud Function
      storage.js           photo upload
    context/
      AuthContext.jsx      who is logged in and their role
    components/            Button, Input, Select, Card, StatusBadge, Table,
Modal, Toast,
                           Tabs, Timeline, Stepper, MapView, Skeleton,
EmptyState,
                           ErrorState, RoleGuard, WrongAppNotice
    hooks/                 useDocument.js, useCollection.js (live reads),
useDebounce.js
    constants/             roles.js, statuses.js, collections.js, appUrls.js
    utils/                 statusRules.js, formatters.js, validators.js,
imageCompress.js
    theme/                 tokens.css, tailwind-preset.js, fonts.css
```

Firebase backend (`firebase/`)

```
firebase/
  functions/
    package.json
    src/
      index.js                lists every Cloud Function
      booking/     getQuote.js, bookDelivery.js, cancelOrder.js
(P1)
      shipments/   updateShipmentStatus.js, saveProof.js,
                   setPaymentStatus.js
(P1)
      staff/       createStaffAccount.js, setUserRole.js
(P1, P2)
      dispatch/    assignShipment.js, reassignShipment.js
(P2)
      live/        copyDriverToShipment.js  (shows the driver card to the
customer)  (P2)
      notifications/  onStatusChange.js, sendSms.js, sendEmail.js
(P3)
      reports/     getReportSummary.js
(P3)
      payments/    startPayment.js, paymentWebhook.js  (optional)
(P3)
      utils/       pricing.js, trackingNumber.js, statusRules.js,
publicTracking.js
  firestore.rules             who can read and write each collection
  storage.rules               who can upload photos, and size and type
limits
  firestore.indexes.json      indexes for filtered lists
```

What connects the three apps

File or folder	What it does	Used by
`packages/shared/src/firebase/firebaseApp.js`	One connection to the same Firebase project	All three apps
`packages/shared/src/context/AuthContext.jsx`	Knows who is logged in and their role	All three apps
`packages/shared/src/constants/roles.js`, `statuses.js`, `collections.js`	The same names for roles, statuses and collections everywhere	All three apps and the functions

File or folder	What it does	Used by
`packages/shared/src/utils/statusRules.js`	Which status moves are allowed (decides which buttons show)	All three apps
`packages/shared/src/components/RoleGuard.jsx`	Hides pages and menu links that the person's role cannot use	Ops dashboard mostly, all three apps
`packages/shared/src/constants/appUrls.js`	Web addresses of the three apps, so a person who logs in to the wrong app sees a link to the right one	All three apps
`packages/shared/src/theme/` and `components/`	The same colours, fonts and building blocks	All three apps
`firebase/firestore.rules`	The access rules for the shared database	All three apps
`firebase/functions/src/`	The protected actions every app calls	All three apps
`firebase/functions/src/utils/statusRules.js`	The server copy of the status rules (must match the shared copy)	Cloud Functions
`firebase.json` and `.firebaserc`	Deploy each app folder to its own Firebase Hosting site	Deployment
Root `package.json`	Lets any app write `import ... from "@parcelo/shared"`	All three apps

The apps never call each other. They connect **through the shared Firebase project**: when the admin changes a status, the customer's order page updates live because both read the same document.

2.3 What happens when a button is pressed

Example: the customer taps **Confirm booking** in the Customer app.

1. The page checks the form is complete.
2. `api/orders.js` calls the `bookDelivery` Cloud Function through the shared `functions.js` helper. The login is sent automatically.
3. The function checks the user is a logged-in customer and that the quote has not expired. It works out the price again itself, so it cannot be tampered with.
4. In one step it creates the order, the shipment with a new tracking number, the first status entry and the public tracking record.
5. It answers with the tracking number and the page shows the success screen.
6. The Ops dashboard's order list updates live, because it listens to the same orders.

2.4 The data (Cloud Firestore)

Collection	Each document holds	From
`users`	name, email, phone, role, notification choices	Phase 1
`quotes`	pickup, dropoff, weightKg, size, distanceKm, price, expiresAt	Phase 1
`orders`	customerId, customerName, addresses, price, paymentMethod, paymentStatus, status, createdAt	Phase 1
`shipments`	orderId, trackingNumber, status, rider name and phone (typed by admin until Phase 2), proof photo link, deliveredAt, failure reason	Phase 1
`shipments/{id}/events`	status, note, changedBy, time (the timeline)	Phase 1
`publicTracking`	the tracking number as its id, status, destination area, timeline	Phase 1
`settings`	price numbers (base fee, per km, per kg, size factors)	Phase 1
`drivers`	userId, name, phone, status, lastLocation, currentShipmentId	Phase 2
`vehicles`	plateNumber, type, maxWeightKg, status, maintenance note	Phase 2
`shipments` (extra fields)	driverId, vehicleId, assignedAt, route, live (driver first name, plate, position)	Phase 2
`products`	name, SKU, quantity, reorderLevel, price	Phase 3
`orders` (extra field)	items: product, quantity, unit price	Phase 3

Collection	Each document holds	From
`notifications`	userId, channel, event, status, attempts, sentAt	Phase 3

Why a separate `publicTracking` collection? A Firestore document is read whole, so fields cannot be hidden. The public tracking page therefore reads a small safe copy (no names, phone numbers or full addresses). The rules let anyone *get* one record by its tracking number but not *list* them all, so numbers cannot be browsed.

How they connect: one customer has many orders; one order has one shipment; one shipment has many events and, from Phase 2, one driver and one vehicle. Documents point to each other by id.

2.5 Statuses

A status may only move along the arrows. The backend refuses any other move.

```
SHIPMENT   Pending -> Assigned -> In Transit -> Out for Delivery -> Delivered
              |                                      |
              |                                      +-> Failed -> (try
again) Out for Delivery
              +-> Cancelled   (only while still Pending)

ORDER      Placed -> Confirmed -> Shipped -> Delivered        (or Cancelled)

PAYMENT    Unpaid -> Paid
           Unpaid -> Pending Transfer -> Paid
```

In Phase 1 there is no driver yet, so the admin moves a shipment straight from *Pending* to *In Transit*. The *Assigned* step switches on in Phase 2.

2.6 Who can do what

Action	Customer	Admin	Dispatcher	Driver	Anyone
Book a delivery, see own orders	Yes				
Look up a tracking number	Yes	Yes	Yes	Yes	Yes
See all orders and shipments		Yes	Yes		
Change shipment status		Yes	Yes	Own jobs only	
Assign drivers and vehicles		Yes	Yes		
Manage drivers, vehicles, users		Yes	Drivers and vehicles		
Manage inventory and reports		Yes			

The backend (security rules and Cloud Functions) checks the role on every request. Hiding a button in the website is only for convenience; the backend check is the real protection.

2.7 Design system

Colours are only part of the design. This section fixes everything a developer needs to make the three apps look like one product: colours, fonts and where each is used, spacing, shapes, buttons, forms, icons, maps and wording.

Colours (the Parcelo dark dashboard theme)

Use	Colour	Hex
Page and sidebar background	Near-black	`#0F172A`
Cards, tables, panels	Dark slate	`#1E293B`
Borders and dividers	Slate	`#334155`
Buttons, active links	Electric teal	`#14B8A6`
Hover and focus	Light teal	`#2DD4BF`
Pressed buttons	Deep teal	`#0D9488`
Main text	Off-white	`#F1F5F9`
Hints and labels	Cool grey	`#94A3B8`

Status badge colours (used in all three apps): Pending amber `#F59E0B`, In Transit blue `#3B82F6`, Out for Delivery purple `#8B5CF6`, Delivered green `#22C55E`, Failed red `#EF4444`. Every badge shows the status word as well as the colour.

Typography: which font goes where

Where it is used	Font	Size	Weight
Page title (one per page)	Space Grotesk	28 px (24 px on phones)	Bold 700
Section heading	Space Grotesk	22 px (20 px on phones)	SemiBold 600
Card title	Space Grotesk	18 px	SemiBold 600

Where it is used	Font	Size	Weight
Price on the quote card	Space Grotesk	36 px	Bold 700
Body text	IBM Plex Sans	16 px, line height 1.5	Regular 400
Sidebar and menu links	IBM Plex Sans	15 px	Medium 500
Buttons	IBM Plex Sans	16 px (14 px for small buttons)	SemiBold 600
Driver app buttons	IBM Plex Sans	18 px	SemiBold 600
Hints, help text, error messages	IBM Plex Sans	13 px	Regular 400
Table headers and form labels	IBM Plex Sans	12 px, capital letters, slightly wider spacing	Medium 500
Status badges	IBM Plex Sans	12 px	SemiBold 600
Tracking numbers, IDs, plate numbers	IBM Plex Mono	14 px	Medium 500

- Load only the weights listed (400, 500, 600, 700) so pages stay light. Use `font-display: swap` so text appears at once.
- Give every font a fallback: Space Grotesk and IBM Plex Sans fall back to `system-ui, sans-serif`; IBM Plex Mono falls back to `ui-monospace, monospace`.
- Save the font files once in `packages/shared/src/theme/` so all three apps use the same ones.

Spacing and shape

Item	Rule
Spacing	Multiples of 4 px (4, 8, 12, 16, 24, 32)
Page padding	16 px on phones, 24 px on desktop
Card padding	16 px on phones, 20 px on desktop
Card corners	12 px

Item	Rule
Button and input corners	8 px
Badge corners	Fully rounded (pill)
Borders	1 px `#334155`
Shadows	None; cards are separated by borders, which suits a dark theme
Focus ring	2 px `#2DD4BF` with a 2 px gap, on every clickable item
Breakpoints	360 px (small phone), 768 px (tablet), 1024 px (desktop). Design for 360 first

Buttons, forms and tables

Part	Style
Primary button	Teal `#14B8A6` background, near-black `#0F172A` text, hover `#2DD4BF`, pressed `#0D9488`
Secondary button	Transparent, 1 px teal border, teal text
Danger button	Red `#EF4444` background, white text (cancel, delete)
Disabled button	50% opacity, no hover
Button height	44 px; 56 px in the Driver app
Input	`#0F172A` background on a `#1E293B` card, 1 px `#334155` border, teal border when focused, red border and a short message when wrong
Table	No background on rows, 1 px divider lines, header in small muted capitals, rows highlight softly on hover
Badge	Colour from the status list, pill shape, word always shown
Loading, empty, error	Every list has all three: grey skeleton blocks, a friendly message with a next step, and an error message with a Retry button

Icons: lucide-react, 20 px (24 px in the Driver app), line width 1.75. An icon never carries meaning alone; it sits beside a word.

Maps: dark map tiles with the OpenStreetMap credit kept visible. The route line is teal, 4 px. Markers: pickup teal, drop-off purple `#8B5CF6`, driver light teal `#2DD4BF`.

Motion: only quick, quiet changes (150 ms on hover and focus) and grey loading skeletons. No large animations.

How each app should feel

App	Feel	Layout
Customer app	Roomy and friendly, phone first	Top bar, one column, a big tracking box on the home page
Ops dashboard (Admin and Dispatcher)	Dense and efficient, desktop first; the map is prominent on the dispatch screens	240 px dark sidebar whose links depend on the role, top bar with search, tables and count cards; the dispatch board has lists on the left, map on the right, live updates
Driver app	Large and simple, one task per screen	Top bar and a bottom bar with three icons, 56 px buttons, high contrast for sunlight

Wording: buttons start with a verb ("Confirm booking", "Assign driver", "Mark delivered"). Error messages say what to do next ("Add a drop-off address"), not just what went wrong. Status words are always the same: Pending, Assigned, In Transit, Out for Delivery, Delivered, Failed, Cancelled.

Accessibility: body text keeps strong contrast against the dark background; every button works with a keyboard; touch targets are at least 44 px; colour is never the only signal.

Where the theme lives: one file, `packages/shared/src/theme/tokens.css`, holds all of the above as variables, and all three apps import it.

```
:root {
  --bg: #0F172A;            --surface: #1E293B;        --border: #334155;
  --accent: #14B8A6;        --accent-hover: #2DD4BF;   --accent-pressed:
#0D9488;
  --text: #F1F5F9;          --text-muted: #94A3B8;
  --status-pending: #F59E0B;   --status-transit: #3B82F6;   --status-out:
#8B5CF6;
  --status-delivered: #22C55E; --status-failed: #EF4444;
  --font-heading: "Space Grotesk", system-ui, sans-serif;
  --font-body: "IBM Plex Sans", system-ui, sans-serif;
  --font-mono: "IBM Plex Mono", ui-monospace, monospace;
  --radius-card: 12px;      --radius-control: 8px;
}
```

2.8 How the apps talk to Firebase

There are two ways an app uses the backend.

1. **Reading data:** the app asks Firestore directly (for example "my orders"). The result updates live. The security rules decide who may read what.
2. **Actions that must be protected:** the app calls a Cloud Function (for example "book this delivery"). The function checks who you are, applies the rules and writes the data. Apps are not allowed to write these records directly.

The tables in each phase list the operations the apps need, written like web addresses (such as `POST /api/orders`). Use this table to see how each one is built in Firebase.

Operation in the phase tables	How it is built in Firebase	Phase
Register, log in, current user	Firebase Authentication; role read from the login	1
Get a quote	Cloud Function `getQuote`	1
Book a delivery	Cloud Function `bookDelivery`	1
List orders, order details	Read `orders` (rules: the owner or staff)	1
Cancel an order	Cloud Function `cancelOrder`	1
Set payment status	Cloud Function `setPaymentStatus` (admin)	1
List shipments, shipment details	Read `shipments` and its `events` (rules: staff)	1
Change shipment status	Cloud Function `updateShipmentStatus`	1

Operation in the phase tables	How it is built in Firebase	Phase
Save rider name and phone	Admin writes the shipment fields (rules: admin only)	1
Save proof photo	Upload to Storage, then Cloud Function `saveProof`	1
Public tracking lookup	Read one `publicTracking/{trackingNumber}` record	1
Waiting for a driver	Read `shipments` where status is Pending	2
Assign, reassign	Cloud Functions `assignShipment`, `reassignShipment`	2
Add a driver or dispatcher	Cloud Function `createStaffAccount` (admin)	2
List and edit vehicles	Read and write `vehicles` (rules: admin and dispatcher)	2
Driver sends position	The driver writes only `lastLocation` on their own `drivers` record (rules)	2
Customer sees driver and position	Cloud Function `copyDriverToShipment` puts the driver's first name, plate and position on the shipment while it is out for delivery; the customer reads their own shipment	2
Products and stock	Read and write `products` (rules: admin); stock changes inside `bookDelivery` and `adjustStock`	3
Messages	Automatic: Cloud Function `onStatusChange` sends and logs them	3
Reports	Cloud Function `getReportSummary` (admin)	3
Online payment (optional)	Cloud Functions `startPayment` and `paymentWebhook`	3

Calling a function from an app (the same helper is used by all three apps):

```
// packages/shared/src/firebase/functions.js
import { getFunctions, httpsCallable } from "firebase/functions";
export const callFunction = (name, data) =>
  httpsCallable(getFunctions(), name)(data).then((result) => result.data);

// apps/customer/src/api/orders.js
export const bookDelivery = (quoteId, paymentMethod) =>
  callFunction("bookDelivery", { quoteId, paymentMethod });
```

A taste of the security rules (`firebase/firestore.rules`):

```
// anyone may look up ONE tracking record, but nobody may list them all or
change them
match /publicTracking/{trackingNumber} { allow get: if true; allow list,
write: if false; }

// a customer reads their own orders; staff read all; nobody writes directly
match /orders/{orderId} {
  allow read: if isStaff() || resource.data.customerId == request.auth.uid;
  allow write: if false;
}

// a driver may change only the lastLocation field of their own record
match /drivers/{id} {
  allow update: if isDriver() && resource.data.userId == request.auth.uid
                &&
request.resource.data.diff(resource.data).affectedKeys().hasOnly(["lastLocati
on"]);
}
```

Errors: Cloud Functions answer with standard codes. The messages shown to people are written in the app.

Code	Meaning	Example
`invalid-argument`	The input is wrong	Destination is missing
`unauthenticated`	Not logged in	Login expired
`permission-denied`	Logged in but not allowed	Customer calls `assignShipment`
`not-found`	Record does not exist	Unknown tracking number
`failed-precondition`	Clashes with the current state	Assigning a driver who is busy, or an invalid status move

Code	Meaning	Example
`internal`	Unexpected problem	Anything else

3. Phase 1: Book, Track and Deliver (Foundation)

3.1 Goal

By the end of Phase 1 a customer can book a delivery, see the price, get a tracking number and follow the package. An admin runs everything by hand from a simple dashboard. This phase is complete on its own: if you stopped here you would still have a working product to present.

3.2 What gets built

Feature	What it does
Login and roles	Customers and admins register and log in; each sees only their own pages
Instant quote	Works out a price from distance, weight and package size
Booking (3 steps)	Pickup and drop-off, package details, then price and confirm
Orders	Customers see their orders; admins see all orders
Shipments	Each booking gets a shipment with a tracking number and a status history
Status updates	Admin moves a shipment forward with one button
Proof of delivery	Admin attaches a photo before a shipment can be marked Delivered
Payment status	Pay on delivery or bank transfer; admin marks it Paid
Public tracking page	Anyone enters a tracking number and sees the timeline

Not in Phase 1: assigning drivers, the driver pages, live location, inventory, messages and reports.

3.3 Screens

App	Page	Web address	Who	What it shows
Customer app	Home	`/`	Anyone	Tracking box and a Book button
Customer app	Tracking result	`/track/:trackingNumber`	Anyone	Status timeline and the delivery photo
Customer app	Login, Register	`/login`, `/register`	Anyone	Forms
Customer app	Book a delivery	`/book`	Customer	The 3-step booking form
Customer app	My orders	`/orders`	Customer	List of orders with status badges
Customer app	Order details	`/orders/:id`	Customer	Price, status, timeline, rider contact
Ops dashboard	Login	`/login`	Admin	Staff login
Ops dashboard	Dashboard	`/`	Admin	Counts: Pending, In Transit, Out for Delivery, Delivered, Failed
Ops dashboard	All orders	`/orders`	Admin	Search by tracking number, filter by status
Ops dashboard	Shipment page	`/shipments/:id`	Admin	Status button, rider name and phone, photo upload, payment status

Booking, step 3 (phone)

```
+---------------------------------+
|  < Back          Step 3 of 3    |
|  [o]------[o]------[*]          |
|                                 |
|  Pickup:    Ikeja, Lagos        |
|  Drop-off:  Lekki, Lagos        |
|  Package:   Medium, 3 kg        |
|                                 |
|  +--------------------------+   |
|  |  Estimated price         |   |
|  |  N 3,900     (about 12 km)|   |
|  +--------------------------+   |
|                                 |
|  Payment:  (o) Pay on delivery  |
|            ( ) Bank transfer    |
|                                 |
|  [      Confirm booking      ]  |
+---------------------------------+
```

Ops dashboard, shipments (desktop)

```
+----------+-------------------------------------------------------------------+
| PARCELO  |  Search tracking number...                        Admin  (A)      |
|          +-------------------------------------------------------------------+
| Overview |  [Pending 12] [In Transit 8] [Out for Delivery 5] [Delivered]      |
| Orders   |                                                                   |
| Shipments|  Tracking       Customer    Status        Payment   Action        |
|          |  TRK-84729301   A. Bello    In Transit    Unpaid    [Next]        |
|          |  TRK-84729302   C. Eze      Pending       Paid      [Next]        |
+----------+-------------------------------------------------------------------+
```

Public tracking page

```
  TRK-84729301                          [ In Transit ]
  Destination: Lekki, Lagos

  (*) Pending               9:00 am
  (*) In Transit           11:30 am
  ( ) Out for Delivery        --
  ( ) Delivered               --
```

3.4 How it works, step by step

Booking

1. The customer enters pickup, drop-off, weight and size. The website asks the server for a quote (`POST /api/quotes`) and shows the price.
2. The customer picks a payment method and taps Confirm (`POST /api/orders`).
3. The server creates the order, then the shipment with a new tracking number and status *Pending*, then the first status entry, all together.
4. The customer sees the tracking number and can copy it.

Updating a shipment

1. The admin opens the shipment and taps the next-status button. Only valid next steps are shown.
2. The website sends `PATCH /api/shipments/:id/status` .
3. The server checks the move is allowed, saves a new status entry with the time and who did it, and updates the shipment and the order.
4. The customer's pages and the tracking page show the new status.

Proof of delivery

1. The admin chooses a photo. The website shrinks it to about 1280 pixels wide.
2. The photo is uploaded to storage and its link is saved with `POST /api/deliveries/:id/proof` .
3. Only now can the shipment be set to *Delivered*. The photo shows on the tracking page.

Tracking

1. A visitor types a tracking number. The website calls `GET /api/track/:trackingNumber` .
2. The server returns only safe details: status, the destination area and the timeline. No names, phone numbers or full addresses.

3.5 Working out the price

```
price = (base fee + price per km x distance + price per kg x weight) x size
factor

base fee = N1,500    per km = N120    per kg = N100
size factor: small 1.0 | medium 1.2 | large 1.5
```

Worked example: 12 km, 3 kg, medium. (1,500 + 1,440 + 300) x 1.2 = 3,888, rounded to the nearest N50 = **N3,900**.

- The numbers are examples. Keep them in the settings collection so the admin can change them without touching code.

- **Distance in Phase 1:** use the straight-line distance between the two points and multiply by 1.3 to approximate the road. Phase 2 replaces this with real route distances.
- Turn addresses into map coordinates with Nominatim, searching when the user presses a button rather than on every key press.
- A quote expires after 15 minutes. The booked price is locked and never changes afterwards.

The price function is a small self-contained piece of code, so it is easy to test: give it 12, 3, medium and it must return 3900.

3.6 API for Phase 1

These are the operations the Customer app and the Ops dashboard need. With Firebase, reads come straight from Firestore and every protected action is a Cloud Function. Section 2.8 shows how each operation below is built.

Method	Address	What it does	Who
POST	/api/auth/register	Create a customer account	Anyone
POST	/api/auth/login	Log in and receive a token	Anyone
GET	/api/users/me	Current user and role	Logged in
POST	/api/quotes	Get an instant price	Customer
POST	/api/orders	Book: creates order, shipment and tracking number	Customer
GET	/api/orders	List orders (customers see their own, admins see all)	Customer, Admin
GET	/api/orders/:id	One order	Owner, Admin
PATCH	/api/orders/:id	Cancel an order (customer only while Pending) or edit (admin)	Owner, Admin
PATCH	/api/orders/:id/payment	Set payment status	Admin
GET	/api/shipments	List shipments, filter by status	Admin
GET	/api/shipments/:id	Details and status history	Admin
PATCH	/api/shipments/:id/status	Move to the next status	Admin
PATCH	/api/shipments/:id/rider	Save rider name and phone	Admin
POST	/api/deliveries/:id/proof	Save the proof photo	Admin

Method	Address	What it does	Who
GET	/api/track/:trackingNumber	Public tracking lookup	Anyone

Your project documentation lists a separate delivery status endpoint. Here a delivery and its shipment share one status endpoint, so the two can never disagree.

Example: get a quote

```
POST /api/quotes
{ "pickup": "Ikeja, Lagos", "dropoff": "Lekki, Lagos", "weightKg": 3, "size":
"medium" }

200 OK
{ "id": "QUO-501", "distanceKm": 12, "price": 3900, "currency": "NGN",
"expiresAt": "2026-11-09T09:15:00Z" }
```

Example: book

```
POST /api/orders
{ "quoteId": "QUO-501", "paymentMethod": "pay_on_delivery" }

201 Created
{ "orderId": "ORD-1001", "shipmentId": "SHP-2001", "trackingNumber": "TRK-
84729301", "shipmentStatus": "pending", "price": 3900 }
```

Example: an invalid move is refused

```
PATCH /api/shipments/SHP-2001/status
{ "status": "delivered" }

409 Conflict
{ "error": { "code": "CONFLICT", "message": "Cannot move from pending to
delivered" } }
```

3.7 Rules the server enforces

- A shipment can only follow the allowed status moves (section 2.5).
- A shipment cannot be set to Delivered without a proof photo.
- A Failed delivery needs a reason and can be tried again.
- A customer can cancel only while the shipment is Pending.
- Customers see only their own orders. Only admins can change status or payment.
- The tracking number is created by the server, is hard to guess, and the public lookup returns no personal details.

- The first admin is created in the Firebase console and given the admin role; the public register form can only create customers.
- Passwords are handled by Firebase Authentication. Photos must be JPG or PNG and under about 2 MB, enforced by the storage rules.

3.8 Outside tools used

Tool	Used for
Leaflet and OpenStreetMap	Showing the pickup and drop-off points on a map
Nominatim	Turning addresses into coordinates
Firebase Storage	Storing proof photos

3.9 Build steps, in order

Step	What to do	You are done when
1	Create the Firebase project and the folders `apps`, `packages/shared` and `firebase`; create the Customer app and the Ops dashboard with Vite; add the theme and fonts to the shared folder	Both apps open as a dark blank page using one shared theme
2	Build the shared components, `AuthContext` and the layouts	A test page shows every component
3	Turn on Firebase Authentication; build register, login and roles (set the first admin by hand)	Customers and admins land in their own apps
4	Build the `getQuote` Cloud Function	12 km, 3 kg, medium gives N3,900
5	Build `bookDelivery` and the booking pages in the Customer app	A booking creates an order, a shipment, a tracking number and a public tracking record
6	Write the first security rules and test them	A customer cannot read another customer's order
7	Build My orders and Order details	A customer sees only their own orders
8	Build the Ops dashboard pages (Dashboard, All orders and Shipment page) with `updateShipmentStatus`	The next-status button shows only valid moves

Step	What to do	You are done when
9	Build the public tracking page	A tracking number shows its timeline
10	Add photo upload (`saveProof`) and payment status	Delivered is refused without a photo
11	Add loading, empty and error states, test on a phone, write the README, deploy both apps to Firebase Hosting	The full path works on the live sites

3.10 Check that it works

- [ ] A new customer can register, book, and receive a tracking number.
- [ ] The price on screen matches the formula.
- [ ] A customer cannot open another customer's order.
- [ ] The admin can move a shipment forward, and cannot skip steps.
- [ ] Delivered is refused without a photo.
- [ ] Anyone can look up a tracking number, and the page shows no personal details.
- [ ] An unknown tracking number shows a friendly message.
- [ ] The booking and tracking pages work on a phone-sized screen.

3.11 What to say and show

Say: "Phase 1 answers the two questions customers care about most: how much will it cost, and where is my package? It works on its own, and everything later builds on the same orders and shipments."

Demo (about 2 minutes): book a delivery and copy the tracking number; switch to the admin dashboard and move the status; upload a photo and mark it delivered; refresh the public tracking page to show the change.

4. Phase 2: Dispatch, Drivers and Fleet (Operations)

4.1 Goal

In Phase 1 the admin updates everything by hand. Phase 2 brings in the people and vehicles that do the work. **A dispatcher assigns a shipment to a driver and a vehicle, the driver runs the job from a phone, and the customer sees who is coming.** Everything from Phase 1 keeps working; Phase 2 only adds to it.

4.2 What gets built

Feature	What it does
Dispatch board	Shows shipments waiting for a driver, free drivers and free vehicles, with an Assign button
Driver management	Admin adds drivers and sees whether each is available, on a job or off duty
Fleet management	Admin adds vehicles and sets them available, in use or in maintenance
Driver pages	A phone-friendly area where a driver sees only their own jobs and taps buttons to update status
Real route	Shows the delivery route and distance on the map
Basic live location	While a package is out for delivery, the driver's phone sends its position so the dispatcher and the customer can see progress
Driver proof photo	The driver takes the delivery photo, replacing the admin upload from Phase 1

Not in Phase 2: automatic assignment, multi-stop route planning, messages and reports.

4.3 What changes for each person

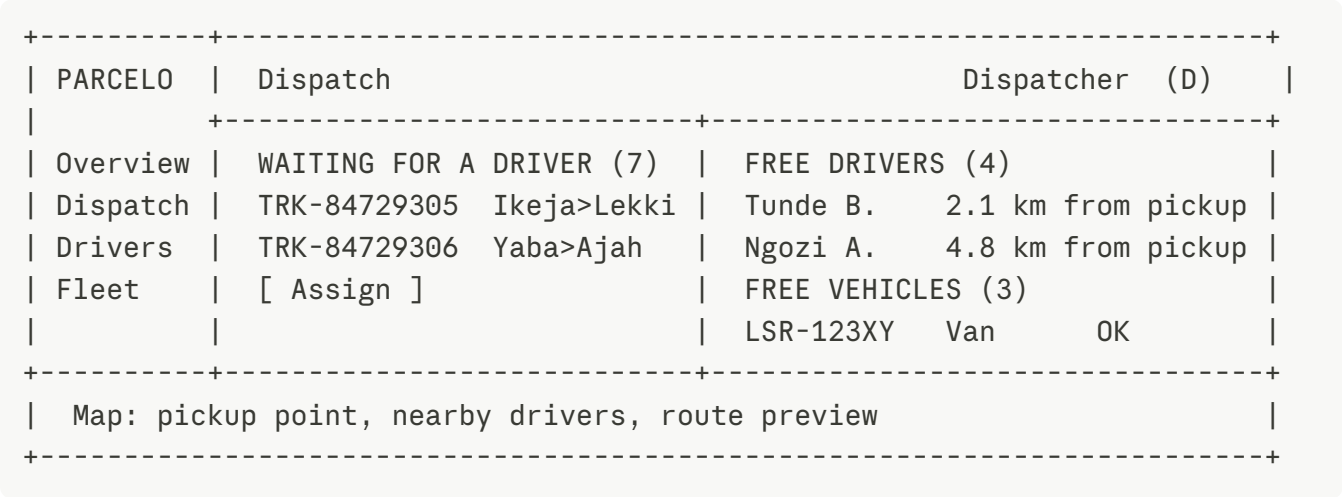
Person	In Phase 2
Dispatcher (new)	Uses the Ops dashboard (dispatch menus): dispatch board, shipments, drivers and fleet
Driver (new)	Logs in on a phone, sees today's jobs, taps Start, Out for Delivery, Delivered or Failed, takes the proof photo
Admin	Same as Phase 1, plus manages drivers and vehicles
Customer	Order page now shows the driver's first name, vehicle and a live map when the package is out for delivery

The rider name and phone that the admin typed in Phase 1 are replaced by the real driver link. Old shipments keep their typed values.

4.4 Screens

App	Page	Web address	Who
Ops dashboard	Dispatch board	`/dispatch`	Dispatcher, Admin
Ops dashboard	All shipments and details	`/shipments`, `/shipments/:id`	Dispatcher, Admin
Ops dashboard	Drivers (Admin can also add and edit)	`/drivers`	Dispatcher, Admin
Ops dashboard	Fleet (Admin can also add and edit vehicles)	`/fleet`	Dispatcher, Admin
Ops dashboard	Staff accounts	`/users`	Admin
Driver app	My jobs	`/`	Driver
Driver app	Job details	`/jobs/:id`	Driver
Driver app	Availability	`/profile`	Driver
Customer app	Order details (updated)	`/orders/:id`	Customer

Dispatch board (desktop)



Driver job screen (phone)

```
+--------------------------------+
|  Job TRK-84729301              |
|  [ map with route ]            |
|  Pickup:   Ikeja     (done)    |
|  Drop-off: Lekki, 12 km        |
|  Customer: A. Bello   [Call]   |
|                                |
|  [    START DELIVERY     ]     |
|  [    MARK DELIVERED     ]     |
|  [    FAILED DELIVERY    ]     |
|  [    Take proof photo   ]     |
|  Sharing location: ON          |
+--------------------------------+
```

4.5 How it works, step by step

Assigning a job

1. A new booking appears on the dispatch board as *Pending*.
2. The dispatcher picks a free driver (the list shows who is nearest to the pickup) and a free vehicle, then taps Assign.
3. The server does all of this in one go, or none of it: checks the shipment is Pending, the driver is available and the vehicle is available; saves the driver and vehicle on the shipment; sets the status to *Assigned*; marks the driver *on job* and the vehicle *in use*.
4. If any check fails the server answers with a 409 message and nothing changes. This is what stops two dispatchers giving the same driver two jobs.
5. The job appears on the driver's phone.

Doing the job

1. The driver taps **Start Delivery**: the status becomes *In Transit*.
2. The driver taps **Out for Delivery** when starting the last leg: the status changes and location sharing starts.
3. The driver takes the proof photo and taps **Delivered**. If the customer is not there, the driver taps **Failed** and gives a reason.
4. When the job ends, the server frees the driver and vehicle and stops location sharing.

Live location

1. While the status is *Out for Delivery*, the phone sends its position about every 20 seconds (`POST /api/drivers/:id/location`).
2. The server keeps the latest position. The dispatcher and the customer who owns the order see the marker move.

3. The public tracking page does **not** show the position.

Reassigning

If a driver cannot continue, the dispatcher uses Reassign. The old driver and vehicle are freed and the new ones are locked in the same single step. Reassigning is blocked once the package is out for delivery.

4.6 Data added in Phase 2

Driver

Field	Notes
id, userId, name, phone	Linked to a login account with the Driver role
status	available, on job or off duty
lastLocation	latitude, longitude and time of the latest ping
currentShipmentId	empty when free

Vehicle

Field	Notes
id, plateNumber	Plate number is unique
type	bike, car or van
maxWeightKg	Used to warn if a package is too heavy
status	available, in use or in maintenance
maintenanceNote, nextServiceDate	Free text and a date

Shipment (new fields): `driverId`, `vehicleId`, `assignedAt`, and `route` (distance in km and time in minutes).

4.7 Dispatch rules

- The board **suggests** drivers sorted by distance to the pickup point. The dispatcher still decides. Automatic assignment is a later improvement.
- A driver has **one active job at a time**. Multi-stop routes are a later improvement.
- A vehicle must be able to carry the package. If the weight is too high, show a warning; the dispatcher can override it with a note.

- A driver who is off duty and a vehicle in maintenance never appear in the pickers.
- When a shipment becomes Delivered, Failed or Cancelled, the driver and vehicle become available again.
- A Failed shipment goes back on the dispatch board to be tried again.
- Drivers can only move **their own** shipments forward one step at a time. Moving backwards or cancelling is for dispatchers and admins, with a note.
- The real route is requested once, when a job is assigned, and saved on the shipment.

4.8 API for Phase 2

These are the operations the Ops dashboard and the Driver app need. Section 2.8 shows how each one is built in Firebase.

Method	Address	What it does	Who
GET	/api/dispatch/unassigned	Shipments waiting for a driver	Dispatcher, Admin
POST	/api/dispatch/assign	Assign a driver and vehicle	Dispatcher, Admin
PATCH	/api/dispatch/:id/reassign	Change driver or vehicle	Dispatcher, Admin
GET	/api/drivers	List drivers	Dispatcher, Admin
POST	/api/drivers	Add a driver	Admin
PATCH	/api/drivers/:id	Edit a driver or set availability	Admin, Driver (own)
GET	/api/drivers/:id/jobs	A driver's jobs	Driver (own), Dispatcher, Admin
POST	/api/drivers/:id/location	Send the driver's position	Driver (own)
GET	/api/vehicles	List vehicles	Dispatcher, Admin
POST	/api/vehicles	Add a vehicle	Admin
PATCH	/api/vehicles/:id	Update status or maintenance	Admin, Dispatcher
PATCH	/api/shipments/:id/status	Now also used by drivers for their own jobs	Driver (own), Dispatcher, Admin
POST	/api/deliveries/:id/proof	Now also allowed for the assigned driver	Driver (own), Admin

Method	Address	What it does	Who
GET	/api/orders/:id/live	Driver card and latest position	Order owner

Example: assign

```
POST /api/dispatch/assign
{ "shipmentId": "SHP-2001", "driverId": "DRV-05", "vehicleId": "VEH-02" }

200 OK
{
  "shipmentId": "SHP-2001",
  "status": "assigned",
  "driver": "Tunde Bello",
  "vehicle": "Toyota Hiace (LSR-123XY)",
  "route": { "distanceKm": 13.4, "durationMin": 31 }
}
```

Example: the driver is already busy

```
409 Conflict
{ "error": { "code": "CONFLICT", "message": "Driver DRV-05 already has an
active job" } }
```

Example: location ping

```
POST /api/drivers/DRV-05/location
{ "lat": 6.5244, "lng": 3.3792 }

204 No Content
```

4.9 Security and privacy

- The Dispatcher and Driver roles are saved on the user's Firebase account, so every request carries the role. A driver can read and change only shipments assigned to them.
- Assignment, status changes and freeing drivers all run on the server, never in the browser.
- Location is collected only while a package is out for delivery, and the driver is told clearly when it is on.
- Only the latest position is shown. Older pings are deleted after 24 hours.

- The customer sees the driver's first name, the vehicle type and plate number, and a Call button that dials without showing the number.
- Every assignment and status change records who did it and when.

4.10 Outside tools used

Tool	Used for	Note
OpenRouteService	Real route distance, time and line	Called from a Cloud Function once per assignment; free tier with daily limits
Nominatim	Addresses to coordinates	Same as Phase 1
Leaflet with a dark map style	Route and moving marker	Keep the OpenStreetMap credit on the map
Browser location (Geolocation)	The driver's position	Needs the driver's permission and https, which Firebase Hosting provides
Firestore live listeners	Instant updates on the dispatch board and the customer's map	Built in; no extra tool needed
Service worker (PWA)	Lets the Driver app be installed and open on a weak signal	Vite PWA plugin

4.11 Build steps, in order

Step	What to do	You are done when
1	Create the Driver app in `apps/`; add the dispatcher and driver roles, the `createStaffAccount` function and the wrong-app notice	Each role reaches only its own app
2	Make the Ops dashboard menu depend on the role; add the Drivers, Fleet and Staff pages	Admin sees every link; a dispatcher sees only the dispatch links
3	Build `assignShipment` as one all-or-nothing step	Assigning the same driver twice is refused

Step	What to do	You are done when
4	Build the dispatch board in the Ops dashboard	A pending shipment becomes Assigned
5	Add the route lookup and save it on the shipment	Distance and time appear on the shipment
6	Build the Driver app: pages, big status buttons and install support (phone first)	A driver completes a whole job on a phone
7	Add the proof photo from the driver's phone	Delivered is refused without a photo
8	Add live location: the driver writes `lastLocation` and `copyDriverToShipment` shares it with the customer	The marker moves while out for delivery and stops afterwards
9	Add the "waiting to send" message and retry for weak signal	A tap made offline is sent when the signal returns
10	Update the security rules for the new roles, test every role on a phone, update the README, deploy the Driver app and the updated Ops dashboard	The full Phase 2 path works on the live sites

4.12 Check that it works

- [ ] A dispatcher can assign a pending shipment to a free driver and vehicle.
- [ ] A busy driver or a vehicle in maintenance cannot be assigned.
- [ ] A driver sees only their own jobs and cannot open another driver's job.
- [ ] A driver cannot skip steps (for example Assigned straight to Delivered).
- [ ] After Delivered, the driver and vehicle are free again.
- [ ] The customer sees the driver card and a moving marker only while the package is out for delivery.
- [ ] The public tracking page shows no live position.

4.13 What to say and show

Say: "Phase 2 turns a record-keeping tool into an operations tool. The dispatcher no longer relies on phone calls, the driver has one simple screen, and the customer knows who is coming. It uses the same shipment data as Phase 1, so nothing is entered twice."

Demo (about 2 minutes): assign a pending shipment on the dispatch board; show the job on a phone-sized window; tap Start Delivery, Out for Delivery and Delivered with a photo; show the customer's order page with the moving marker. Finally try to assign the same driver again to show the conflict message.

5. Phase 3: Inventory, Notifications and Reports (Expansion)

5.1 Goal

Phase 3 completes the system. It connects **stock** to orders so the business knows what it has, **tells customers** what is happening without anyone sending messages by hand, and gives the business **simple reports**. It builds on the same orders and shipments from Phases 1 and 2.

5.2 What gets built

Feature	What it does
Inventory	Product list with stock levels and a reorder level; admin adds products and adjusts stock
Stock reduction	When an order that contains products is confirmed, the stock goes down automatically
Low-stock alerts	Products at or below their reorder level are flagged
Notifications	SMS or email when a shipment is assigned, out for delivery or delivered
Notification settings	Customers can choose SMS, email or both
Reports and charts	Orders per week, delivery success rate and low-stock items
Online payment (optional)	Paystack or Flutterwave for card and bank payments
Outside carrier tracking (optional)	Pulls status from carriers such as 17TRACK for parcels your own drivers do not carry

Two kinds of order. A *delivery order* moves a package the customer already has (what Phase 1 does). A *product order* contains items from the business's own stock, so it also uses inventory. Both create a shipment in exactly the same way.

5.3 What changes for each person

Person	In Phase 3
Admin	Manages products and stock, reads reports, sees the message log
Customer	Receives messages, can choose how, and can order products if the business sells them
Dispatcher and Driver	No new screens

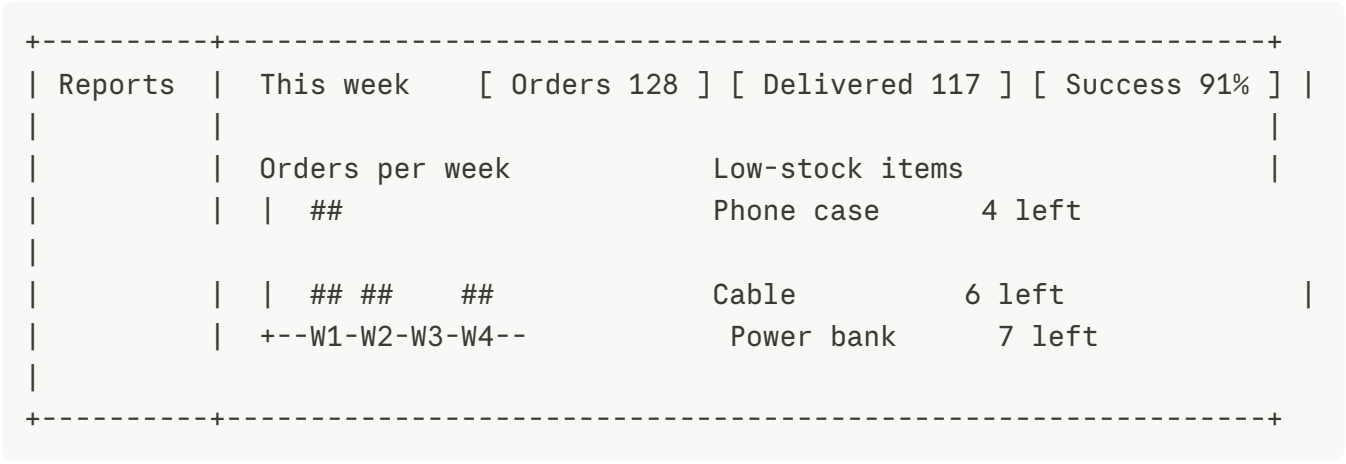
5.4 Screens

App	Page	Web address	Who
Ops dashboard	Inventory list	`/inventory`	Admin
Ops dashboard	Product details and stock adjust	`/inventory/:id`	Admin
Ops dashboard	Reports dashboard	`/reports`	Admin
Ops dashboard	Message log	`/notifications`	Admin
Ops dashboard	Settings (prices and reorder defaults)	`/settings`	Admin
Customer app	Notification settings	`/settings/notifications`	Customer

Inventory page (desktop)

```
+----------+-------------------------------------------------------------+
| PARCELO  |  Inventory                          [ + Add product ]  Admin |
|          +-------------------------------------------------------------+
| Overview |  [ Low stock (3) ]  [ All (48) ]          Search...          |
| Orders   |  SKU      Product         Qty    Reorder   Status             |
| Inventory|  SK-001   Phone case       4      10       ! Low stock       |
| Reports  |  SK-002   Charger         52      15       OK                |
+----------+-------------------------------------------------------------+
```

Reports dashboard (desktop)



5.5 How it works, step by step

Stock for product orders

1. A customer places an order that contains products.
2. When the order is confirmed, the server checks every item has enough stock. If any item is short, the whole order is refused with a clear message.
3. If all items are available, the quantities go down in the same single step that creates the order.
4. If a product falls to its reorder level or below, it is flagged on the Low stock tab.
5. If the order is cancelled while the shipment is still Pending, the quantities go back up.

Notifications

1. Whenever a shipment gets a new status entry, the server looks up the message for that event.
2. It sends an SMS or email to the customer, using their chosen channel.
3. It saves a log entry (sent or failed). A failed message is retried up to three times.

Event	Example message
Booking confirmed	"Parcelo: order ORD-1001 booked. Track it with TRK-84729301."
Assigned	"Your driver Tunde has been assigned."
Out for delivery	"Your package is out for delivery."
Delivered	"Your package has been delivered."

Messages never contain full addresses.

Reports

The reports page asks the server for totals over a date range (`GET /api/reports/summary`). The server counts orders, delivered shipments and failed shipments, works out the success rate, and lists low-stock products. The website draws the numbers as cards and charts.

5.6 Data added in Phase 3

Record	Fields	Notes
Product	id, name, SKU, quantity, reorderLevel, price, active	SKU is unique
Order item	orderId, productId, quantity, unitPrice	Only on product orders
Notification	id, userId, channel (sms or email), event, status (sent, failed), attempts, sentAt	The message log
User (added)	notifyBySms, notifyByEmail, phone	Customer's choices

If online payment is added: a **Payment** record with orderId, provider, reference, amount, status and paidAt.

5.7 Rules

- Stock can never go below zero.
- A product order is all or nothing: if one item is short, nothing is booked.
- A product is low when `quantity <= reorderLevel` .
- Customers can turn off optional messages, but delivery updates are always sent.
- Reports are for admins only and show totals, not personal details.
- If online payment is added, an order is marked Paid only after the payment provider confirms it to the server. The customer's browser returning to the site is never treated as proof of payment.

5.8 API for Phase 3

These are the operations the Ops dashboard and the Customer app need in this phase. Section 2.8 shows how each one is built in Firebase.

Method	Address	What it does	Who
GET	/api/products	List products and stock	Admin

Method	Address	What it does	Who
POST	/api/products	Add a product	Admin
PATCH	/api/products/:id	Edit a product or adjust stock	Admin
GET	/api/products/low-stock	Products at or below the reorder level	Admin
GET	/api/notifications	Message log	Admin
PATCH	/api/users/me/notifications	Choose SMS, email or both	Customer
GET	/api/reports/summary	Totals for the dashboard	Admin

Example: low stock

```
GET /api/products/low-stock

200 OK
{ "data": [ { "id": "PRD-17", "name": "Phone case", "quantity": 4,
"reorderLevel": 10 } ] }
```

Example: report summary

```
GET /api/reports/summary?from=2026-11-01&to=2026-11-07

200 OK
{ "orders": 128, "delivered": 117, "failed": 6, "successRate": 91.4,
"lowStockCount": 3 }
```

Example: a product order is refused

```
POST /api/orders
{ "items": [ { "productId": "PRD-17", "quantity": 6 } ] }

409 Conflict
{ "error": { "code": "CONFLICT", "message": "Not enough stock for Phone case
(4 left)" } }
```

5.9 Security

- Only admins can see or change products, reports and the message log.
- Phone numbers and emails are used only for sending messages.
- API keys for SMS, email and payments are stored as secrets in Cloud Functions, never in the app code.

- If payment is added: card details are entered only on the payment provider's page, and the server checks the provider's signature before trusting a payment confirmation.

5.10 Outside tools used

Tool	Used for
Termii (Nigeria) or Twilio	SMS messages
Resend or SendGrid	Email messages
Recharts	Charts on the reports page (loaded only on that page)
Paystack or Flutterwave (optional)	Online card and bank payments
17TRACK or AfterShip (optional)	Outside carrier tracking

5.11 Build steps, in order

Step	What to do	You are done when
1	Build the product records and the inventory pages	Admin can add and edit products
2	Add stock reduction to product orders (all or nothing)	A short order is refused and stock stays the same
3	Add low-stock flags and the Low stock tab	A product at its reorder level appears there
4	Build the message sender and the first template, then the other three	A status change produces a message and a log entry
5	Add notification settings for customers	Switching a channel off stops those messages
6	Build the reports endpoint and the reports page	The numbers match the real data
7	Optional: add online payment in test mode	A test payment marks the order Paid once
8	Optional: add outside carrier tracking	A carrier tracking number shows its status

Step	What to do	You are done when
9	Test the whole path, update the README, deploy	Book, assign, deliver, notify and report all work together

5.12 Check that it works

- [ ] Placing a product order reduces stock.
- [ ] An order asking for more than is in stock is refused as a whole.
- [ ] Low-stock products are flagged.
- [ ] Cancelling a Pending product order puts the stock back.
- [ ] Each status change sends one message and writes one log entry.
- [ ] Turning off a channel stops those messages.
- [ ] The report totals match the real orders.
- [ ] A customer cannot open the inventory or reports pages.

5.13 What to say and show

Say: "Phase 3 makes the system complete. Orders now affect stock, customers are kept informed without anyone sending messages by hand, and the business can see how it is doing. This is what turns a delivery tracker into a full logistics platform."

Demo (about 2 minutes): place a product order and show the stock number go down; show the low-stock flag; change a shipment's status and show the message in the log; open the reports page.

6. Putting It All Together

6.1 The three phases side by side

	Phase 1	Phase 2	Phase 3
Theme	Foundation	Operations	Expansion
In one line	Book and track	Assign and deliver	Stock, messages and reports
People	Customer, Admin	Dispatcher, Driver	Admin, Customer
New records	Quote, Order, Shipment, Status entry, Delivery	Driver, Vehicle	Product, Order item, Notification

	Phase 1	Phase 2	Phase 3
Main new pages	Booking, Orders, Ops dashboard, Tracking	Dispatch, Drivers, Fleet, Driver pages	Inventory, Reports, Message log
Outside tools	Leaflet, Nominatim, Firebase Storage	OpenRouteService, Firestore live updates	Termii, Resend, Recharts
Question it answers	How much, and where is my package?	Who is delivering it?	What stock is left, and has the customer been told?

6.2 Suggested presentation order (about 8 to 10 minutes)

1. The problem: scattered tools, no tracking, no price, no proof of delivery (1 min).
2. The big picture and the journey of one package, using section 1 (1 min).
3. Phase 1 with the live demo (2 min).
4. Phase 2 with the dispatch and phone demo (2 min).
5. Phase 3 with the stock and message demo (2 min).
6. Tech stack, design and what comes next (1 min).

If someone asks: *Why phases?* Each phase gives something usable, lowers risk, and lets the next phase reuse what already exists. *Why this stack?* React and Firebase work well together, there is no server to run, and the free tiers remove cost barriers. *Where is the database touched?* The apps read it directly, but only what the security rules allow; every important change goes through a Cloud Function.

6.3 Final checklist

- [ ] Phase 1 works end to end from a phone-sized window
- [ ] Phase 2 conflict message (assigning a busy driver) can be shown on request
- [ ] Anything not built yet is clearly shown as planned
- [ ] Colours, fonts and the favicon are consistent across the app and the documents
- [ ] The README explains how to run the three apps and deploy the Firebase backend
- [ ] You can explain each phase in one sentence